

CSE 464: Software QA and Testing

Project Part #3 — README

Amir Boshra

Arizona State University

Spring 2026

GitHub Repository: <https://github.com/aboshra11/CSE-464-2026-aboshra>

Pull Request: <https://github.com/aboshra11/CSE-464-2026-aboshra/pull/1>

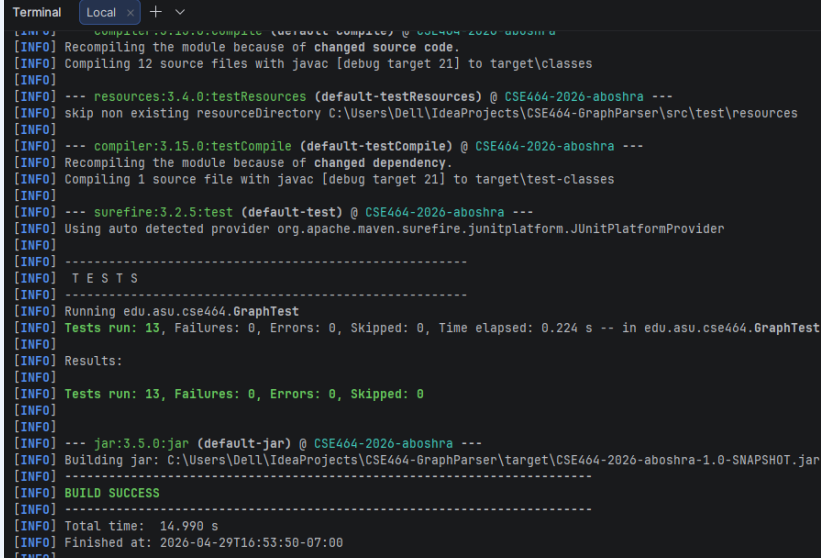
1. How to Compile and Run

Prerequisites: Java JDK 21, Maven, Graphviz must be installed and on PATH.

Compile & Run All Tests

```
| mvn package
```

Compiles the project and runs all 13 JUnit tests automatically.



```
Terminal Local x + v
[INFO] --- compiler:3.10.0:compile (default-compile) @ CSE464-2026-aboshra ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 12 source files with javac [debug target 21] to target\classes
[INFO] --- resources:3.4.0:testResources (default-testResources) @ CSE464-2026-aboshra ---
[INFO] skip non existing resourceDirectory C:\Users\Dell\IdeaProjects\CSE464-GraphParser\src\test\resources
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ CSE464-2026-aboshra ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug target 21] to target\test-classes
[INFO] --- surefire:3.2.5:test (default-test) @ CSE464-2026-aboshra ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running edu.asu.cse464.GraphTest
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.224 s -- in edu.asu.cse464.GraphTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.5.0:jar (default-jar) @ CSE464-2026-aboshra ---
[INFO] Building jar: C:\Users\Dell\IdeaProjects\CSE464-GraphParser\target\CSE464-2026-aboshra-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 14.990 s
[INFO] Finished at: 2026-04-29T16:53:50-07:00
[INFO]
```

2. Five Refactorings

All refactorings were applied on the refactor branch, each as a separate commit.

Refactoring 1 — Rename Method

Type: Rename Method

Why: The method `GraphSearch` violated Java naming conventions which require methods to start with a lowercase letter. Renamed to `graphSearch` for consistency.

- **Commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7dd79aa>

Refactoring 2 — Extract Method: `validateNodes`

Type: Extract Method

Why: The null and existence checks for `src` and `dst` nodes were duplicated inline. Extracted into a private `validateNodes()` method to improve readability and allow reuse.

```
private boolean validateNodes(Node src, Node dst) {  
    return src != null && dst != null &&  
           nodes.containsValue(src) && nodes.containsValue(dst);  
}
```

- **Commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/892eb3a>

Refactoring 3 — Extract Method: `getNeighbors`

Type: Extract Method

Why: Both BFS and DFS repeated the same edge traversal logic to find neighboring nodes. Extracted into `getNeighbors(Node node)` to eliminate duplication.

```
private List<Node> getNeighbors(Node node) {  
    List<Node> neighbors = new ArrayList<>();  
    for (Edge edge : edges) {  
        if (edge.getSource().equals(node))  
            neighbors.add(edge.getDestination());  
    }  
    return neighbors;  
}
```

- **Commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/6250114>

Refactoring 4 — Replace Conditional with Switch Expression

Type: Simplify Conditional

Why: The if-else chain selecting between BFS and DFS was verbose. Replaced with a Java switch expression for clarity and easier extensibility when adding new algorithms.

```
return switch (algo) {
    case BFS -> new BFSSearch();
    case DFS -> new DFSSearch();
    case RANDOM -> new RandomWalkSearch();
};
```

- **Commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/c76dd1c>

Refactoring 5 — Improve toString

Type: Improve toString

Why: Node.toString() returned only the label which lacked context. Updated to Node{label} format to match standard Java conventions and align with the expected output format in Part 3 (e.g., visiting Path{nodes=[Node{a}]}).

```
@Override
public String toString() {
    return "Node{" + label + "}";
}
```

- **Commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/bbf6650>

3. Template Pattern

The Template Pattern was applied by creating an abstract base class GraphSearchTemplate that defines the skeleton of the search algorithm. BFSSearch and DFSSearch extend this class and implement only the steps that differ between the two algorithms.

Common Steps (in GraphSearchTemplate)

- search(Node src, Node dst) — the template method defining the algorithm skeleton
- getNeighbors(Node node) — finds all neighbors via edges (same for BFS and DFS)
- buildCurrentPath(Node current) — reconstructs the path from src to current node
- setEdges(Set<Edge> edges) — injects graph edges into the searcher

Different Steps (implemented by subclasses)

- initialize(Node src) — BFS uses a Queue; DFS uses a Stack
- hasNext() — checks if the frontier is empty
- getNext() — BFS polls from queue (FIFO); DFS pops from stack (LIFO)
- isVisited(Node node) — BFS checks parentMap; DFS checks visited Set
- addToFrontier(Node node, Node parent) — adds to queue or stack
- getParent(Node node) — returns parent for path reconstruction

Class Structure

```

GraphSearchTemplate (abstract)
    implements GraphSearchStrategy
    |-- BFSSearch    (Queue-based)
    |-- DFSSearch    (Stack-based)
    |-- RandomWalkSearch (Random neighbor selection)

```

```

57
58      // Abstract methods – subclasses implement the differences
59      protected abstract void initialize(Node src); 2 usages 3 implementations Amir Boshra
60      protected abstract boolean hasNext(); 2 usages 3 implementations Amir Boshra
61      protected abstract Node getNext(); 2 usages 3 implementations Amir Boshra
62      protected abstract boolean isVisited(Node node); 1 usage 3 implementations Amir Boshra
63      protected abstract void addToFrontier(Node node, Node parent); 1 usage 3 implementations Amir Boshra
64      protected abstract Node getParent(Node node); 1 usage 3 implementations Amir Boshra
65  }

```

[Screenshot: GraphSearchTemplate.java showing abstract methods]

```

10  public class BFSSearch extends GraphSearchTemplate { 1 usage Amir Boshra

```

[Screenshot: BFSSearch.java extending GraphSearchTemplate]

- **Template Pattern commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7645bfb>

4. Strategy Pattern

The Strategy Pattern was applied by creating a GraphSearchStrategy interface. GraphSearchTemplate implements this interface. The graphSearch method in Graph.java selects the correct strategy at runtime based on the Algorithm enum parameter.

Interface

```

public interface GraphSearchStrategy {
    void setEdges(Set<Edge> edges);
    Path search(Node src, Node dst);
}

```

How Algorithm Selection Works

```

GraphSearchStrategy strategy = switch (algo) {
    case BFS    -> new BFSSearch();
    case DFS    -> new DFSSearch();
    case RANDOM -> new RandomWalkSearch();
};
strategy.setEdges(edges);
return strategy.search(src, dst);

```

This means adding a new algorithm only requires creating a new class that extends `GraphSearchTemplate` and adding a case to the switch — no other changes needed.

```
5 public interface GraphSearchStrategy { 2 usages 4 implementations Amir Boshra
6     /**
7      * Must be called before search() to provide the graph edges.
8      */
9     void setEdges(Set<Edge> edges); 1 usage 1 implementation Amir Boshra
10
11     /**
12      * Searches for a path from src to dst.
13      * Returns null if no path exists.
14      */
15     Path search(Node src, Node dst); 1 usage 2 implementations Amir Boshra
16 }
```

[Screenshot: GraphSearchStrategy.java interface]

- **Strategy Pattern commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7cba4ec>

5. Random Walk Search

`RandomWalkSearch` implements both the Template Pattern (extends `GraphSearchTemplate`) and the Strategy Pattern (via `GraphSearchStrategy` interface). At each step, it picks a random neighbor of the current node instead of using a queue or stack.

Algorithm

- Start at src node
- At each step, get all neighbors of the current node
- Pick one neighbor at random
- Move to that neighbor and repeat
- Return the path when dst is reached
- Return null if no neighbors exist or max iterations (1000) is reached

Example Usage

```
Node src = graph.getNodes().get("a");
Node dst = graph.getNodes().get("c");
```

```
Path p1 = graph.graphSearch(src, dst, Algorithm.RANDOM);
Path p2 = graph.graphSearch(src, dst, Algorithm.RANDOM);
Path p3 = graph.graphSearch(src, dst, Algorithm.RANDOM);
```

Example Output (shows randomness across 3 runs)

```
visiting Node{a}
visiting Node{b}
visiting Node{c}
Path: Node{a} -> Node{b} -> Node{c}

visiting Node{a}
visiting Node{e}
visiting Node{g}
visiting Node{b}
visiting Node{c}
Path: Node{a} -> Node{e} -> Node{g} -> Node{b} -> Node{c}
```

```
"C:\Program Files\Java\jdk-21.0.10\bin\java.exe" "-javaagent:D:\CSE 464\Project Part 1\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=57493" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
Number of nodes: 12
Node labels: a b c d e f g h x y z m
Number of edges: 12
Edges:
Node(a) -> Node(b)
Node(b) -> Node(c)
Node(c) -> Node(d)
Node(d) -> Node(a)
Node(a) -> Node(e)
Node(e) -> Node(f)
Node(f) -> Node(g)
Node(g) -> Node(h)
Node(h) -> Node(i)
Node(i) -> Node(j)
Node(j) -> Node(k)
Node(k) -> Node(l)
Node(l) -> Node(m)
Node(m) -> Node(a)

Graph output saved to graph_output.txt
DOT graph saved to exported_graph.dot
PNG graph saved to exported_graph.png

--- Random Walk Run 1 ---
visiting a
visiting a -> e
visiting a -> e -> f
visiting a -> e -> f -> h
Result: null

--- Random Walk Run 2 ---
visiting a
visiting a -> b
visiting a -> b -> c
Result: a -> b -> c

--- Random Walk Run 3 ---
visiting a
visiting a -> b
visiting a -> b -> c
Result: a -> b -> c

GraphParser > src > main > java > edu > asu > cse464 > @Main 53:2 CRLF UTF-8 4 spaces
```

[Screenshot: Random Walk output showing 3 runs with different paths]

- **Random Walk commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/d6296f0>

6. Pull Request & Code Review

A pull request was created from the refactor branch to master. Five review comments were added, responded to, and addressed with code changes.

Pull Request Link

- **Pull Request #1:** <https://github.com/aboshra11/CSE-464-2026-aboshra/pull/1>

Review Comments and Responses

Comment 1 — Null checks in `GraphSearchTemplate.search()`

Review: Consider adding null checks for src and dst at the start of search().

Response: Added null checks at the start of the search method in GraphSearchTemplate.

Comment 2 — Infinite loop in `RandomWalkSearch`

Review: The random walk may run indefinitely on cyclic graphs. Consider adding a max iteration limit.

Response: Added a max iteration limit of 1000 to prevent infinite loops on cyclic graphs.

Comment 3 — Simplify `validateNodes`

Review: validateNodes could be simplified to a single return statement.

Response: Simplified validateNodes to a single return statement.

Comment 4 — Separate visited Set in `BFSSearch`

Review: Using parentMap to track visited nodes in BFS works but a separate visited Set would be clearer.

Response: Added a separate visited Set in BFSSearch for clarity and consistency with DFSSearch.

Comment 5 — Document `setEdges` in `GraphSearchStrategy`

Review: setEdges must be called before search but this is not documented.

Response: Added Javadoc to GraphSearchStrategy clarifying that setEdges must be called before search.



aboshra11 commented [39 minutes ago](#)

[View reviewed changes](#)

aboshra11 left a comment

[Owner](#) [Author](#) ...

Several improvements needed: null checks, infinite loop prevention, code simplification, consistency improvements, and better documentation.

▼ [src/main/java/edu/asu/cse464/GraphSearchTemplate.java](#)

Hide resolved



aboshra11 [42 minutes ago](#)

[Owner](#) [Author](#) ...

Consider adding null checks for src and dst parameters at the beginning of the search method to make the base class more robust and self-contained.



aboshra11 [38 minutes ago](#)

[Owner](#) [Author](#) ...

Good point. Added null checks at the start of the search method in GraphSearchTemplate.

▼ [src/main/java/edu/asu/cse464/RandomWalkSearch.java](#)

Hide resolved



aboshra11 [42 minutes ago](#)

[Owner](#) [Author](#) ...

The random walk may run indefinitely on cyclic graphs since visited nodes can be revisited. Consider adding a max iteration limit to prevent infinite loops.



aboshra11 [36 minutes ago](#)

[Owner](#) [Author](#) ...

Good catch. Added a max iteration limit of 1000 to prevent infinite loops on cyclic graphs.

src/main/java/edu/asu/cse464/RandomWalkSearch.java

Hide resolved



aboshra11 42 minutes ago

Owner

Author

...

The random walk may run indefinitely on cyclic graphs since visited nodes can be revisited. Consider adding a max iteration limit to prevent infinite loops.



aboshra11 36 minutes ago

Owner

Author

...

Good catch. Added a max iteration limit of 1000 to prevent infinite loops on cyclic graphs.

src/main/java/edu/asu/cse464/Graph.java

Hide resolved



aboshra11 41 minutes ago

Owner

Author

...

The validateNodes method could be simplified to a single return statement: `return src != null && dst != null && nodes.containsValue(src) && nodes.containsValue(dst);`



aboshra11 35 minutes ago

Owner

Author

...

Agreed. Simplified validateNodes to a single return statement.

src/main/java/edu/asu/cse464/BFSSearch.java

Hide resolved



aboshra11 41 minutes ago

Owner

Author

...

Using parentMap to track visited nodes in BFS works correctly since every node is only added once, but consider using a separate visited Set for clarity and consistency with DFSSearch.



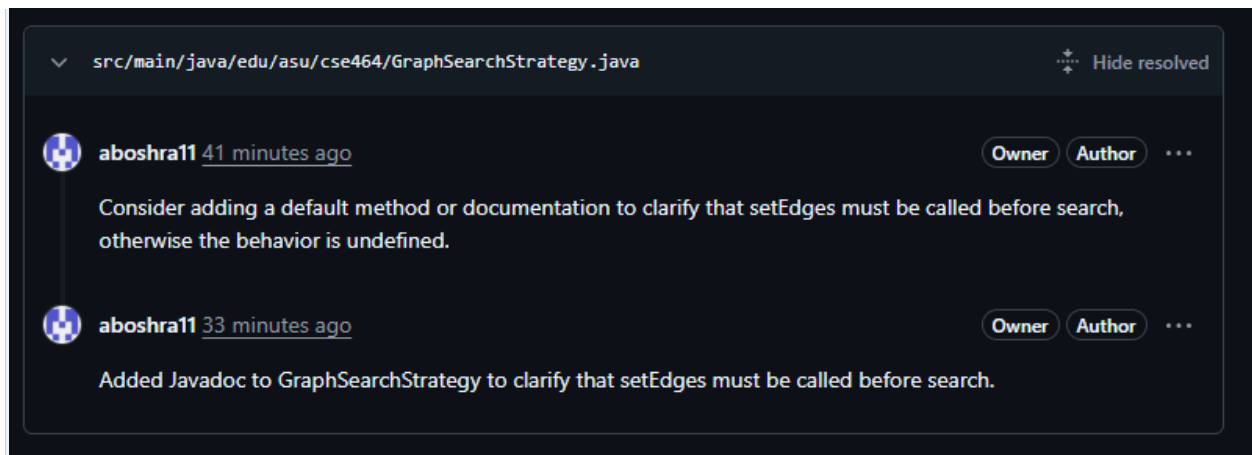
aboshra11 34 minutes ago

Owner

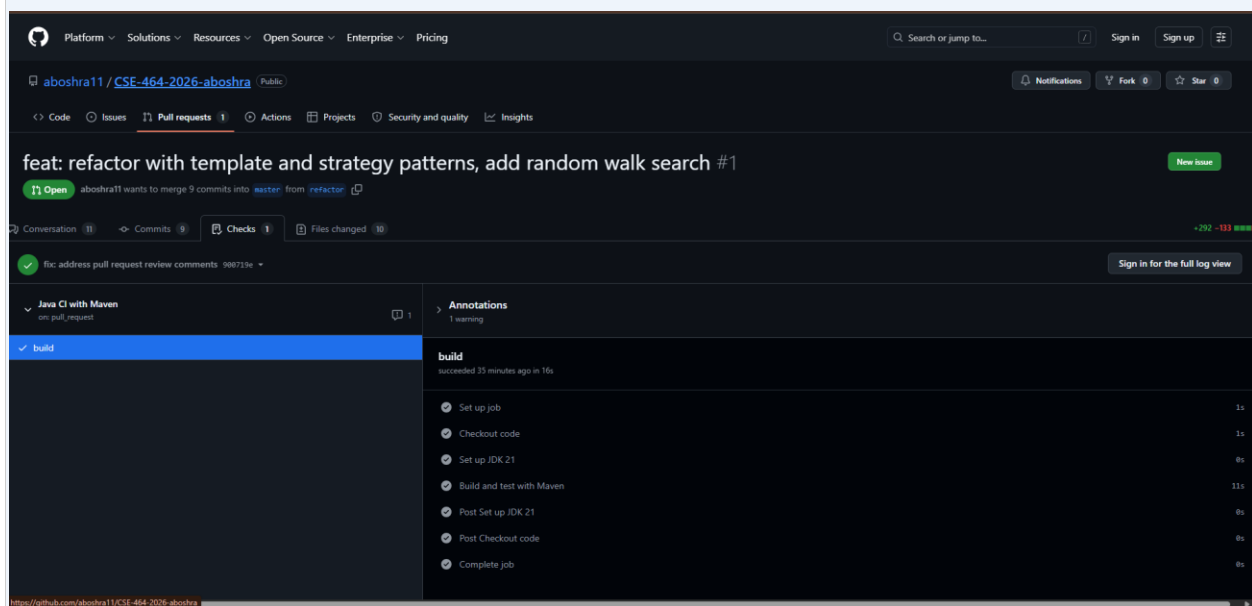
Author

...

Good suggestion. Added a separate visited Set in BFSSearch for clarity.



[Screenshot: GitHub PR showing 5 review comments and responses]



[Screenshot: GitHub PR showing All checks passed green]

- **Review fixes commit:** <https://github.com/aboshra11/CSE-464-2026-aboshra/commit/900719e>

7. Summary of All Commits

Feature	Commit	Link
Rename GraphSearch	7dd79aa	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7dd79aa
Extract validateNodes	892eb3a	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/892eb3a
Extract getNeighbors	6250114	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/6250114
Switch expression	c76dd1c	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/c76dd1c

Feature	Commit	Link
Improve Node toString	bbf6650	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/bbf6650
Template Pattern	7645bfb	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7645bfb
Strategy Pattern	7cba4ec	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/7cba4ec
Random Walk	d6296f0	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/d6296f0
PR review fixes	900719e	https://github.com/aboshra11/CSE-464-2026-aboshra/commit/900719e